

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — CODING CHALLENGE QUESTIONS: 8085 ALP SOLUTIONS

Name :HASMATH NISHA .M

REGNO:192524513

Question 1 — Sort Warehouse Stock, Store Minimum

Problem: Eight storage racks hold quantities in memory (2000H–2007H). Sort them in ascending order, store the sorted list, and identify the rack with the lowest stock.

Algorithm / Logic

- Use Bubble Sort: compare each pair of adjacent bytes and swap if the left one is larger.
- Repeat for 7 passes ($n - 1$) over 8 elements until fully sorted.
- Copy the sorted array from the working area (2000H) to the output area (2100H).
- Since the array is now ascending, the first byte of the sorted array is the minimum — store it at 2200H.

8085 ALP

```
MVIB,07H    ;Outerloop counter = n-1= 7
OUTER: MVIC,07H    ;Innerloop counter (resets each pass)
    LXIH,2000H    ;Point to start of data
INNER: MOVA,M      ;Load current element
    INXH
    MOVD,M        ;Load next element
    CMPD          ;Compare A (current) with D (next)
    JC SKIP       ;If A < D, already in order
    MOVN,M,A      ;Swap: put A in next position
    DCXH
    MOVN,D        ;Put D in current position
    INXH
SKIP: DCRB
    JNZ INNER
    DCRB
    JNZ OUTER
    ;Copy sorted array to 2100H
    LXIH,2000H
    LXID,2100H
    MVIB,08H
COPY: MOVA,M
```

```
STAXD
INXH
INXD
DCRB
JNZCOPY
; Minimum = first element of sorted array
LDA 2000H
STA 2200H
HLT
```

Sample Input / Output (Simulated Trace)

Input at 2000H–2007H: 05H, 02H, 09H, 01H, 07H, 03H, 08H, 04H

Address	Before Sort	After Sort (2000H)	Copied Output (2100H)
2000H	05H	01H	01H
2001H	02H	02H	02H
2002H	09H	03H	03H
2003H	01H	04H	04H
2004H	07H	05H	05H
2005H	03H	07H	07H
2006H	08H	08H	08H
2007H	04H	09H	09H

Result: 2200H = 01H → Minimum quantity (rack with lowest stock).

Load me at

1

MVI B, 07H ; Outer loop counter = n-1 = 7

2

LXI H, 2000H ; Point to start of data

3

4

OUTER: MVI C, 07H ; Inner loop counter

5

LXI H, 2000H

6

7

INNER: MOV A, M ; Load current element

8

INX H

9

MOV D, M ; Load next element

10

CMP D ; Compare A (current) with D (next)

11

JC SKIP ; If A < D, no swap needed

12

MOV M, A ; Swap: put A in next position

13

DCX H

14

MOV M, D ; Put D in current position

15

INX H

16

17

SKIP: DCR C

18

JNZ INNER

19

20

DCR B

21

JNZ OUTER

22

23

; Copy sorted array to 2100H

24

LXI H, 2000H

25

LXI D, 2100H

26

MVI B, 08H

27

COPY: MOV A, M

28

STAX D

29

INX H

30

INX D

31

DCR B

32

JNZ COPY

33

34

; Minimum is now the first element of sorted array

35

LDA 2000H

36

STA 2200H

37

38

HLT

DataStackKeyPadMemory

Start2000

Address (Hex)	Address	Data
07D0	2000	5
07D1	2001	2
07D2	2002	9
07D3	2003	0
07D4	2004	0
07D5	2005	0
07D6	2006	0
07D7	2007	0
07D8	2008	0
07D9	2009	0
07DA	2010	0
07DB	2011	0
07DC	2012	0
07DD	2013	0

Line No	Assembler Message
0	Program assembled successfully

Question 2 — Compare Copied Data Block, Count Mismatches

Problem: An 8-byte block is copied from a source region to a destination region. Each destination byte must be verified against the corresponding source byte, and the number of mismatches counted.

Algorithm / Logic

- Copy 8 bytes one at a time from source (2000H) to destination (2100H) using the accumulator as a relay register.
- Re-scan both blocks in a second loop, comparing each source–destination pair with CMP.
- Increment a counter (register C) whenever a comparison is unequal (mismatch).
- Store the final mismatch count at 2200H.

8085 ALP

```
LXIH,2000H ;Source pointer
LXID,2100H ;Destination pointer
MVI B,08H ;8 bytes to copy
MVIC,00H ;Mismatch counter = 0
COPY: MOV A,M ;Read source byte
STAX D ;Write to destination
INX H
INX D
DCR B
JNZ COPY
;Compare source vs destination
LXIH,2000H
```



```
LXID,2100H
MVIB,08H
CMPLP: LDAXD      ;Load destination byte
MOVE,A
MOVA,M      ;Load source byte
CMPE
JZ MATCH
INRC      ;Increment mismatch count
MATCH: INXH
INXD
DCRB
JNZCMPLP
MOVA,C
STA 2200H    ;Store mismatch count
HLT
```

Sample Input / Output (Simulated Trace)

Source (2000H–2007H): 11H,22H,33H,44H,55H,66H,77H,88H — assume destination bytes at 2107H and 2103H get corrupted after copy to demonstrate detection.

Address	Source Byte	Destination Byte	Match?
2000H/2100H	11H	11H	Yes
2001H/2101H	22H	22H	Yes
2002H/2102H	33H	33H	Yes
2003H/2103H	44H	45H	No—mismatch
2004H/2104H	55H	55H	Yes
2005H/2105H	66H	66H	Yes
2006H/2106H	77H	77H	Yes
2007H/2107H	88H	80H	No—mismatch

Result: 2200H = 02H → Two mismatches detected.

Registers

00	00	00	00	00	00	00	00
01	00	00	00	00	00	00	00
02	00	00	00	00	00	00	00
03	00	00	00	00	00	00	00
04	00	00	00	00	00	00	00
05	00	00	00	00	00	00	00
06	00	00	00	00	00	00	00
07	00	00	00	00	00	00	00
08	00	00	00	00	00	00	00
09	00	00	00	00	00	00	00
0A	00	00	00	00	00	00	00
0B	00	00	00	00	00	00	00
0C	00	00	00	00	00	00	00
0D	00	00	00	00	00	00	00
0E	00	00	00	00	00	00	00
0F	00	00	00	00	00	00	00

Flag

S	0
Z	1
AC	0
P	1
C	0

Decimal - Hex Conversion

Decimal:

Hex:

» To Hex

« To Dec

IO Ports

-

+

Update Port Value

Memory

-

+

Update Memory

Load me at

1

LXI H, 2000H ; Source pointer

2

LXI D, 2100H ; Destination pointer

3

MVI B, 08H ; 8 bytes to copy

4

MVI C, 00H ; Mismatch counter = 0

5

6

COPY: MOV A, M ; Read source byte

7

STAX D ; Write to destination

8

INX H

9

INX D

10

DCR B

11

JNZ COPY

12

13

; Now compare source vs destination

14

LXI H, 2000H

15

LXI D, 2100H

16

MVI B, 08H

17

18

CMPLP: LDAX D ; Load destination byte

19

MOV E, A

20

MOV A, M ; Load source byte

21

CMP E

22

JZ MATCH

23

INR C ; Increment mismatch count

24

25

MATCH: INX H

26

INX D

27

DCR B

28

JNZ CMPLP

29

30

MOV A, C

31

STA 2200H ; Store mismatch count

32

33

HLT

Data

Stack

Keypad

Start

2000

Address (Hex)	Address	Data
07D0	2000	11
07D1	2001	22
07D2	2002	33
07D3	2003	44
07D4	2004	55
07D5	2005	66
07D6	2006	77
07D7	2007	88
07D8	2008	0
07D9	2009	0
07DA	2010	0
07DB	2011	0
07DC	2012	0
07DD	2013	0

Line No	Assembler Message
0	Program assembled successfully

Question 3 — Parity Check of an 8-bit Code

Problem: An 8-bit binary code is valid only if it contains an even number of 1s (even parity). Count the set bits, determine parity, and store a validity flag.

Algorithm / Logic

- Load the byte and test each bit by rotating it into the LSB position (RRC) 8 times.
- After each rotation, mask with ANI 01H to check if the LSB is 1; if so, increment the bit counter (register C).
- Store the total set-bit count at 2100H.
- Test the LSB of the count itself: 0 = even (valid), 1 = odd (invalid). Store 01H (valid) or 00H (invalid) at 2101H.

8085 ALP

```
LDA 2000H ;Load the 8-bit code
MOVB, A ;Save code in B
MVIC, 00H ;Bit counter = 0
MVID, 08H ;Loop 8 times (8 bits)
BITLP: MOVA, B
```

```
ANI01H      ;Isolate current LSB
JZ SKIP
INRC        ;Increment count if bit = 1
SKIP: MOV A,B
RRC         ;Rotate right; next bit moves to LSB
MOVB,A
DCRD
JNZBITLP
MOVA,C
STA2100H     ;Store the set-bit count
ANI01H      ;Check if count is odd or even
JZ VALID
MVA,00H      ;Odd number of 1s -> invalid
STA2101H
JMP DONE
VALID: MVA,01H ;Even number of 1s -> valid
STA2101H
DONE: HLT
```

Sample Input / Output (Simulated Trace)

Input at 2000H: 10110110B (B6H) → Bits set: 1,0,1,1,0,1,1,0 = 5 ones

Rotation #	Byte (B) After RRC	LSB Tested	Running Count (C)
Start	10110110	—	0
1	01011011	0	0
2	10101101	1	1
3	11010110	1	2
4	01101011	0	2
5	10110101	1	3
6	11011010	1	4
7	01101101	0	4
8	10110110	1	5

Result: 2100H = 05H (5 oes — odd) → 2101H = 00H → Code is INVALID (odd parity).

Load me at

1

MVI B, 07H

; Outer loop counter = n-1 = 7

2

LXI H, 2000H

; Point to start of data

3

4

OUTER: MVI C, 07H

; Inner loop counter

5

LXI H, 2000H

6

7

INNER: MOV A, M

; Load current element

8

INX H

9

MOV D, M

; Load next element

10

CMP D

; Compare A (current) with D (next)

11

JC SKIP

; If A < D, no swap needed

12

MOV M, A

; Swap: put A in next position

13

DCX H

14

MOV M, D

; Put D in current position

15

INX H

16

17

SKIP: DCR C

18

JNZ INNER

19

20

DCR B

21

JNZ OUTER

22

23

; Copy sorted array to 2100H

24

LXI H, 2000H

25

LXI D, 2100H

26

MVI B, 08H

27

COPY: MOV A, M

28

STAX D

29

INX H

30

INX D

31

DCR B

32

JNZ COPY

33

34

; Minimum is now the first element of sorted array

35

LDA 2000H

36

STA 2200H

37

38

HLT

Data

Stack

KeYPad

Memory

Start2000

Address (Hex)	Address	Data
07D0	2000	5
07D1	2001	2
07D2	2002	9
07D3	2003	0
07D4	2004	0
07D5	2005	0
07D6	2006	0
07D7	2007	0
07D8	2008	0
07D9	2009	0
07DA	2010	0
07DB	2011	0
07DC	2012	0
07DD	2013	0

Line No	Assembler Message
0	Program assembled successfully